# CMP-142 Object Oriented Programming
# Lab 06

---

**Topics:**

- Aggregation and Composition
- Friend Functions
- Stream Operations (input/output with << and >>)
- Operator Overloading

## Instructions!

1. This is a **graded lab**, you are strictly **NOT** allowed to discuss your solutions with your fellow colleagues, even not allowed asking how is he/she is doing, it may result in negative marking. You can **ONLY** discuss with your TAs or instructor.
2. Indent your code properly.
3. Use meaningful variable and function names. **Use the camelCase notation**.
4. Save your work frequently. Make a habit of pressing CTRL+S after every line of code you write.
5. **Do NOT use any global or static variable(s)**. However, global named constants may be used.
6. Make sure that there are **NO dangling pointers or memory leaks** in your programs.
7. **Make sure that all functions have been implemented and can be called in the main function. Otherwise, no marks will be awarded for that function.**

## [Task 01] Implement the Restaurant Management System

Your task is to create a basic Restaurant Management System that revolves around some classes like the Menu, Items, Orders and Tables. Each of the class has some attributes. Now look at the structure of each class of the system one by one:

1. **Item**

**Private Attributes:**

`name (string)`: The name of the item

`price (double)`: Price of the item

`category (string)`: Category of the item (e.g., "Appetizer", "Main Course", "Dessert")

**Public Member Functions:**

```
Item()

Item(string, double)

friend ostream& operator<<(ostream&, Item&);

friend istream& operator>>(istream&, Item&);

string getName() const

double getPrice() const
```

2. **Menu**

**Private Attributes:**

`items` (array of Item objects): A collection of items available in the restaurant

**Public Member Functions:**

`void addItem(const Item&):` Adds a new item to the menu

`void displayMenu() const:` Displays the entire menu

3. **Order**

**Private Attributes:**

`itemsOrdered` (vector of Item): A list of items ordered

`tableNumber (int)`: The table that placed the order

**Public Member Functions:**

`Order& operator += (const Item&):` Adds an item to the order

`double calculateTotal() const:` Calculates the total cost of the order

`void displayOrder() const:` Displays details of the order

4. **Table**

**Private Attributes:**

`tableNumber (int):` The number of the table

`order (Order):` The order for the table (Composition)

**Public Member Functions:**

`void placeOrder(const Order&):` Places an order for the table

`void displayTableOrder() const:` Displays the order details for the table.

5. **Restaurant**

**Private Attributes:**

`name (char array):` Name of the restaurant

`menu (Menu):` The restaurant's menu

`tables (Array of Table):` A collection of tables in the restaurant

`void addTable(const Table&)`: Adds a new table to the restaurant

`void addMenuItem(const Item&)`: Adds a new item to the menu

`void placeOrder(int tableNumber, const Order&)`: Places an order for a specific table

`void displayRestaurantInfo() const`: Displays the name of the restaurant, menu, and orders for each table.

**Public Member Functions:**

Now, moving onto the task, you need to implement input/output stream overloading for the `Item` class to facilitate the efficient creation and display of menu items. Overload the `+=` operator within the `Order` class to enable the addition of items to an order. Employ aggregation to link the `Menu` class with the `Restaurant` class, allowing a restaurant to maintain a shared, updatable menu. Utilize composition to associate each `Table` with a corresponding `Order`, indicating that each table can place a unique order.

Implement friend functions to enable access to private data members, particularly within the `Item` class, for overloading the `<<` and `>>` stream operators. Include methods to display relevant information for each class, as follows:

1. Display the menu using the `Menu` class.
2. Display the details of an order using the `Order` class.
3. Display comprehensive restaurant information, including tables and their respective orders, using the `Restaurant` class.

# CMP-142 Object Oriented Programming
# Lab 06

## [Task 02] Implement the University Department Management System

Develop a C++ program that models a university department management system by implementing Department and Course classes. Each Department instance should encapsulate an array of Course objects, while each Course object should maintain a dynamic array of enrolled student names. The solution must showcase the use of advanced C++ object-oriented programming (OOP) concepts, including constructors, copy constructors, destructors, dynamic memory management, aggregation and composition, friend functions, and operator overloading (relational and stream operators).

***NOTE: The prototype of the functions is not provided. You are on your own determining the prototypes.***

***Use Dynamic Memory Allocation for arrays.***

### 1. Class: Course

**Private Member Variables:**

std::string courseName — Represents the course's name.

int courseID — A unique identifier for the course.

std::string* students — A dynamically allocated array for storing the names of enrolled students.

int numStudents — The number of enrolled students.

**Public Member Functions:**

Parameterized constructor for initializing courseName, courseID, and dynamic allocation of the students array.

Copy constructor for deep copying to manage dynamic memory correctly.

Destructor to release dynamically allocated memory.

### 2. Class: Department

**Private Member Variables:**

std::string departmentName — Represents the department's name.

Course* courses — A dynamically allocated array of Course objects.

int numCourses — The number of courses in the department.

# CMP-142 Object Oriented Programming
# Lab 06

**Public Member Functions:**

| |
|---|
| Parameterized constructor for initializing departmentName, numCourses, and dynamic allocation for the courses array.<br><br>Copy constructor for deep copying Department objects to ensure proper memory management.<br><br>Destructor to handle the deallocation of dynamic memory.<br><br>Overloaded << stream operator for formatted output of department details, including all associated courses.<br><br>Overloaded relational operators (<, >, ==) for comparing Department objects based on the number of courses. |

**Functional Requirements:**

A. Implement dynamic memory management to ensure correct handling of arrays in both Course and Department classes.
B. Implement relational operator overloading (<, >, ==) in the Department class for comparing departments based on the number of courses.
C. Overload the << stream operator in both classes for formatted output of object details.
D. Utilize aggregation and composition to model the relationship between Department and Course.

**Demonstration Requirements in *main():***

1. Instantiate multiple Course objects, each with a dynamically allocated array of student names.
2. Create Department objects that aggregate an array of Course objects.
3. Demonstrate the display of Course and Department details using the overloaded << stream operator.
4. Use the overloaded relational operators to compare and display results for Department objects.
5. Validate the functionality of copy constructors and assignment operators by creating copies of Course and Department objects and confirming correct behavior.
6. Ensure that destructors handle dynamic memory deallocation properly to prevent memory leaks.